

DataShield Algorithm Guide

Algorithm-level reference for the DataShield codec: packet format, encoding and decoding logic, error-correction math, sector accumulation, targeted rebinding, and file assembly with version-collision resolution, plus the randomized test bench. The document covers algorithms and data structures only; the public API and solution layout are described in `DeveloperGuide.en.md`. Each of the three tasks has its own document family in three levels of detail (a strict academic monograph, an engineering retelling with matching section numbering, and a from-scratch tutorial): encoding — `EncoderGuide.Academic/Engineer/Tutorial.en.md`, decoding — `DecoderGuide.Academic/Engineer/Tutorial.en.md`, assembly and collisions — `AssemblyGuide.Academic/Engineer/Tutorial.en.md` (invariants, the mathematics of exhaustive search and rotation, worked examples, guarantee boundaries).

1. Purpose and Principles

DataShield is a forward error correction (FEC) codec for transferring small files over unreliable one-way channels: text pastes, messengers, e-mail bodies, raw byte streams. A file becomes a stream of small self-contained packets; the decoder rebuilds the file from an arbitrarily damaged, reordered, duplicated, and multi-file stream.

Goal	Mechanism
Text transport	Packet = 75 bytes = exactly 100 Base64 chars, no padding
Loss resilience	Reed–Solomon erasure code over GF(2 ¹⁶) at the volume level
Arbitrary arrival order	Orderless accumulation; a sector carries only its own number
Noise resilience	Sliding-window scanning with truncated SHA-256 per packet
Multi-file streams	Sector-to-header binding through an H5-seeded hash
Forgeries and collisions	Payload versions with confirmation counts + correct-combination search at assembly

The core design idea: **no byte of the input stream is ever treated as garbage until it fails a cryptographically strong checksum**. Everything else follows from this.

2. Packet Format

Every packet has the same size — 75 bytes (`PacketFormat.PacketSize`). In text mode a packet is Base64-encoded: $75 \cdot 4/3 =$ exactly **100 characters with no padding**; one line = one packet.

2.1. Header packet (H)

Field	Offset	Size	Contents
H1	0	14	File name (packed, see below), ASCII, space-padded
H2	14	3	File size, low 3 bytes of UInt32, LE
H3	17	32	SHA-256 of the file contents
H4	49	2	ECC volume count M, UInt16 LE
H5	51	24	Trunc24(SHA-256(H1–H4)) — 192 bits

The H1 field stores not the full name but a packed one (`FileNameCodec.Pack`): the name is split at the **first** dot; the extension (including compound ones like ".tar.gz") is kept intact while the base is truncated with a trailing `~` marker (`documents.tar.gz` → `docume~.tar.gz`). If less than 2 bytes remain for the base, the name is unrepresentable and encoding is rejected. The full name is recovered via the SHA-256 (H3) by the file's owner.

2.2. Data sector (D)

Field	Offset	Size	Contents
D1	0	2	Sector (volume) number, UInt16 LE
D2	2	64	Payload — file data or ECC
D3	66	9	Trunc9(SHA-256(H5 D1 D2)) — 72 bits

2.3. The two-level hash — the heart of the format

- **The header hash (H5) is autonomous.** Trunc24(SHA-256(H1–H4)) is verified with no context. This lets the scanner recognize a header in a noisy stream "from scratch" (false positive probability $\sim 2^{-192}$).
- **The sector hash (D3) is header-bound.** It is computed as Trunc9(SHA-256(H5 | D1 | D2)): the seed is H5 of the owning header. Without knowing H5 the check is impossible — random 75 bytes will almost certainly not pass it ($\sim 2^{-72}$).

Consequences:

1. A sector is **cryptographically bound to its file**. In a multi-file stream a sector cannot be confused with someone else's header: the hash only matches its own H5.
2. A sector arriving **before** its header is not recognized on the first pass (H5 is unknown yet). It is picked up later by a targeted rebinding of retained data (section 8).
3. The volume count N is derived from H2 (`ceil(FileSize / 64)` , minimum 1), so H4 defines the full number space: sectors are numbered `0 .. N+M-1` (first the N data volumes, then the M ECC volumes).

3. File Encoding

`FileEncoder.Encode(content, fileName)` performs the following steps (progress scale: preparation 0–10, ECC 10–75, packets 75–100):

1. **Validation.** Size $\leq 16,777,215$ bytes (3-byte H2 field); the name is packed into the H1 field (14 bytes): the base is truncated with a `~` marker, the extension is kept intact; an unrepresentable name is rejected.
2. **SHA-256** of the contents — the future assembly verifier.
3. **Splitting into data volumes:** $N = \max(1, \lceil \text{size} / 64 \rceil)$; the last volume is zero-padded.
4. **ECC volume count:** $M = \max(1, \lceil N \cdot \text{eccPercent} / 100 \rceil)$ when $\text{eccPercent} \geq 1$, otherwise 0. Field limit: $N + M \leq 65,535$.
5. **RS encoding:** N data volumes $\rightarrow M$ ECC volumes (section 4).
6. **Header:** serialize H1–H4 (51 bytes) $\rightarrow \text{Trunc24}(\text{SHA-256}) \rightarrow$ header packet. The H5 value is remembered.
7. **Sectors:** for each volume $i \in [0, N+M)$ a packet `seqNum(2) + payload(64) + hash(9)`, where the hash is computed with seed = H5.
8. **Stream placement:** header copies $H = \max(3, \lceil (N+M) \cdot \text{headerPercent} / 100 \rceil)$; the first and last packets of the stream are headers, intermediate copies are inserted evenly, every `max(1, (N+M) / (H-1))` sectors.
9. **Wipe** of intermediate buffers (data/ECC payloads, serialized header).

The result is a list of 75-byte packets. The text form (`EncodeToText`) is one Base64 line per packet.

3.1. Header Placement Details

`ArrangePackets` builds the stream as follows: the first packet is a header; then the sectors follow in number order, and after every `interval`-th sector an intermediate copy is inserted, where `interval = max(1, (N+M) / (H-2+1))`; the last packet is a header again. The integer division is deliberate: for small T the copies gravitate to the start of the stream, which is safer against tail truncation. The total number of copies is always exactly H (intermediate ones are inserted while the `H-2` counter lasts). The sector order in the stream is natural (0, 1, ..., $N+M-1$) — the decoder is order-agnostic, and predictability helps debugging and stream concatenation.

3.2. End-to-End Encoding Example

A 100-byte file `report.txt`, ECC 10%, headers 3%:

1. The name `report.txt` (9 ASCII bytes) fits into H1 without truncation — space-padded to 14.
2. $N = \lceil 100/64 \rceil = 2$; volume D1 holds bytes 64..99 plus 24 zero padding bytes.
3. $M = \max(1, \lceil 2 \cdot 10 / 100 \rceil) = 1$; $T = N+M = 3 \leq 65,535$ — the field suffices.
4. RS: for each of the 32 symbol positions s a column `[d0s, d1s, e0s]` is assembled; `Process` computes $e0^s$, an ECC-volume symbol (the coefficients are fixed by the RsRaid16 code matrix).
5. Header: H1(14) + H2(3: `100` $\rightarrow$ `64 00 00`) + H3(32: the file's SHA-256) + H4(2: `01 00`) $\rightarrow$ 51 bytes $\rightarrow$ H5 = `Trunc24(SHA-256(...))` $\rightarrow$ the 75-byte header packet H.

- Sectors: D-packets for numbers 0 (data), 1 (data), 2 (ECC); each hash = $\text{Trunc9}(\text{SHA-256}(\text{H5} \parallel \text{number} \parallel \text{payload}))$.
- $H = \max(3, \lceil 3 \cdot 3 / 100 \rceil) = 3$; interval = $\max(1, 3/2) = 1 \rightarrow$ a copy after the first sector.
- Stream (6 packets): `H, D0, H, D1, D2, H`.
- Wipe: the data/ECC payload buffers and the serialized header are cleared; only the returned packets live on.

4. Redundancy: the Reed–Solomon erasure code

The $\text{GF}(2^{16})$ field lives in `RsRaid16` (never modified; see the `RsRaid16Demo` demo). The adapter `RsC` `odecAdapter` operates on 64-byte volumes:

- One field symbol = 2 bytes (UInt16 LE). A volume = **32 independent GF symbols**.
- Encoding:** for each of the 32 symbol positions a column of K data symbols is assembled, `RsRaidBase.Process` appends M ECC symbols to the tail of the column, and the results are written into the ECC volumes. Each ECC volume is a linear combination of the data volumes over the field.
- Decoding (erasures, not errors):** the input is K+M slots with a validity map. The recovery condition is **the number of erased data volumes ≤ the number of available ECC volumes**. The system matrix is inverted with the erasure map; erased symbols are recovered position by position.
- Limit: $K + M \leq 65,535$ (field size).

Why an erasure code rather than error correction: the integrity of every received volume is already guaranteed by truncated SHA-256 (missing a forgery with probability $\sim 2^{-72}$). The codec only needs to restore **gaps**, not fix distortions — radically simpler and faster.

The practical file-size limit comes from the field, not H2: without ECC $N \leq 65,535 \rightarrow \sim 4.19$ MB; at `eccPercent = 10%`, $N \cdot 1.1 \leq 65,535 \rightarrow \sim 3.8$ MB.

4.1. Position-by-Position Encoding Mechanics (K=2, M=1)

The volumes `D0`, `D1`, and `E0` carry 32 GF symbols each (UInt16 LE). Encoding walks the positions `s = 0..31` independently:

Position <code>s</code>	Column before <code>Process</code>	Column after <code>Process</code>
0	<code>[d0⁰, d1⁰, ?]</code>	<code>[d0⁰, d1⁰, e0⁰]</code>
1	<code>[d0¹, d1¹, ?]</code>	<code>[d0¹, d1¹, e0¹]</code>
...	...	...
31	<code>[d0³¹, d1³¹, ?]</code>	<code>[d0³¹, d1³¹, e0³¹]</code>

Every ECC symbol is the same linear form of the column's K data symbols with the code-matrix coefficients; the matrix is built by `RsRaid16.Init(k, m, null)` and does not depend on the data. Recovery is symmetric: `Init(k, m, validityMap)` builds the system from the valid slots only, and `Process` solves it for each column separately. To the adapter, `RsRaid16` is a black box with the "Init →

per-column Process" contract; the field computations themselves (log tables, matrix inversion) live in the unmodifiable reference `refs-src/RsRaid16.cs`.

Cost: encoding — 32 columns × K field additions/multiplications per ECC symbol; recovery — a single matrix inversion at `Init` plus 32 columns × a system solve. The work is linear in the file size.

5. The Decoding Pipeline

```
byte source (Source)
    |
    ▼ (text mode only: drops everything outside the Base64 alphabet)
ByteRangeFilter
    |
    ▼ (sliding window with a 1-byte step; retains the whole stream)
SlidingWindowScanner
    | EmitPacket: only recognized 75-byte packets
    ▼
StreamProcessor accumulator → ReceptionSlot × number of files
    |
    ▼ (on demand: TryAssemble)
assembly: direct → RS → version-collision search
```

- **Source** (`ByteArraySource`, `StreamSource`, `FileSource`) — event-driven buffered model (see `DeveloperGuide.en.md`).
- **Filter** is included in text mode only: it turns the stream into a "dense" Base64 byte stream, discarding line breaks, spaces, and junk. In binary mode the filter is excluded from the chain.
- **Scanner** — see section 6.
- **Accumulator** — see section 7.

Two input formats are supported: **Base64 text** (100-byte window) and **raw packets** (75-byte window). Scanners for both formats are created lazily and live side by side: a mixed stream (text + embedded binary chunks) is handled by sequential `Scan` calls; retained data persists across calls.

6. Recognition in Noise: the Sliding Window

`SlidingWindowScanner` applies a window delegate `WindowHandler(ReadOnlySpan<byte> window, out byte[]? emitted) → int` to stream positions. The value returned by the delegate is the advance of the **direct pass**:

- the window at position p is recognized → return `emitted` (a copy of the packet) and an advance of the packet length (75 or 100 bytes);
- not recognized → advance **by 1 byte**; the window slides.

The post-success jump by the packet length is a direct-pass optimization. It can skip the start of another valid packet overlapping the one just accepted (this actually happens on damage: a fragment of one volume together with the first characters of a neighboring line accidentally decodes into a valid packet). Coverage completeness does not depend on the jumps: the rebinding re-scan (section 8) checks the retained data at **every** position.

Window logic (text mode `FileDecoder.TxtWindow`): decode 100 Base64 chars into 75 bytes (decode failure → shift by 1); ask the accumulator `Recognizes(packet)` (no side effects) — that is the autonomous header hash **or** the sector hash under one of the known headers. Success → emit the packet.

Thus desynchronization, noise, and line breaks between packets are handled automatically: the window "feels out" the next packet boundary by shifting byte by byte.

6.1. Direct-Pass Trace (text mode, fragment)

```
stream: ...junk...[100 Base64 chars of a packet][junk]...
position p, window [p, p+100):
  p      - the window starts in junk: decode/recognition fails → advance +1
  p+1    - same → +1
  ...    - byte-by-byte shifts until the window lands on the packet boundary
  p+k    - window = packet: Base64 → 75 B, Recognizes = true
          → EmitPacket(a copy), advance +100
  p+k+100- the window after the packet (junk) → +1, the search continues
```

The direct pass costs $O(\text{stream length})$ window invocations in the worst case; on a dense stream (packets back to back) — $O(\text{length}/75)$ thanks to the jumps. The post-success +100 jump can skip the start of a packet overlapping the accepted one (this actually happens under damage) — such losses are closed by the exhaustive rebinding re-scan (section 8): it never jumps at all.

The scanner **retains the entire processed stream** (input volumes are small) — the price of targeted rebinding (section 8): data unrecognized on the first pass is never lost.

7. Sector Accumulation

A two-level structure: `StreamProcessor` is the outer loop (stream of chunks → packets → slots); `ReceptionSlot` is the elementary accumulator of a single file.

7.1. StreamProcessor: from chunks to packets

Input arrives in arbitrary chunks. The `_pending[75]` buffer stitches chunks together: bytes are gathered up to a full packet in a loop; an incomplete tail stays in `_pending` until the next chunk. A full packet is cloned and goes through `AcceptPacket`:

1. **Autonomous hash matched** → a header packet → `AcceptHeader`.

2. **Otherwise** → try to accept as a sector for **every** known slot (`AcceptSectorForSlot`): the number is in `[0, N+M)` and the hash seeded with that slot's H5 matches. One packet can theoretically be a valid sector of several slots — it is accepted into all of them.

A recognized packet is emitted down the pipeline (`EmitPacket` — indivisible portions). New slots are collected into a list, and **after leaving the lock** `HeaderAccepted(header, headerHash)` is raised for each — a signal for upper modules to perform a targeted rebind (section 8).

Header acceptance `AcceptHeader`:

- byte-wise comparison with every existing slot → a match means another copy: `IncrementHeaderCount()` (saturating at `int.MaxValue`), no new slot;
- otherwise — deserialize `HeaderContent`, read H5 from the packet, create a `ReceptionSlot`, add it to the list.

Thread safety: reception and state reads (snapshots `Slots`, counters) share one lock; events are raised outside it.

7.2. ReceptionSlot: a single file's slot

Internal state:

```
SortedDictionary<int sectorNum, List<SectorVariant>> _sectors
```

- the key is the sector number `[0, N+M)`;
- the value is payload **versions**; a `SectorVariant` = `payload(64)` + `ConfirmationCount` (how many copies of this version were received);
- the version list is **always sorted by descending count** — the first element is the most confirmed one.

`AddSector(sectorNum, payload)` step by step:

1. Number outside `[0, N+M)` or length $\neq 64$ → reject.
2. First version of the sector? Create the list, add a version with count 1.
3. Linear search for a version with a **byte-equal** payload:
 - found → `ConfirmationCount++` (saturating), then **bubbling up**: the version rises while the neighbor above has a strictly smaller count. It is not swapped past equal counts — this preserves the current order of equally-probable elements (important for the assembly heuristic, section 9.7).
 - not found → a new version is appended **to the end** (all existing ones have count ≥ 1 , so the sort stays valid).

Why versions instead of "last one wins": a channel may duplicate and delay packets. If two different copies of the same number pass the hash check (an extremely unlikely but finite event — or a deliberate forgery), the accumulator keeps **both** with their counters. Reception is lossless: the decision of which version is correct is postponed until assembly, where an independent arbiter exists — the SHA-256 of the whole file.

Slot metrics: `HeaderReceptionCount` (header copies), `ReceivedSectorCount` (numbers with ≥ 1 version), `ReceivedSectorCopyCount` (all copies, sum of counters), `CollisionSectorCount` (numbers with > 1 version), `Coverage` (share of received numbers of $N+M$), `BuildValidityMap()` (a ) map: received/collision/missing), `BuildCollisionMap()` (number $\rightarrow$ collision multiplicity), `GetSectorVersions(n)` (a snapshot of versions in preference order).

7.3. Multiple files

Every unique header spawns its own slot. The H5-seeded hash prevents a sector from leaking into someone else's slot: even identical volume numbers of different files differ by H5. Assembly (`FileDecoder.TryAssemble(header)`) finds the slot by byte-comparing the serialized header and assembles exactly that one.

8. Targeted Rebinding: a Sector Before Its Header

The problem: a sector that arrives before the first header of its file is not recognized on the first pass — H5 is unknown, and the hash check is impossible (an autonomous header-style hash over D1–D2 almost never matches D3). The scanner shifts by 1 byte and moves on.

The solution: on `HeaderAccepted` the decoder asks **both** scanners (`txt` and `bin`) to run `RequestRescan` with the special `RebindWindow`:

1. The window is decoded as usual (Base64 $\rightarrow$ 75 bytes, or raw 75 bytes).
2. The sector number is checked against the `[0, N+M)` range of the new header.
3. The sector hash seeded with the **new** header's H5 is verified.
4. Success $\rightarrow$ the packet is emitted (accepted by the accumulator as a sector of this slot), failure $\rightarrow$ nothing; in both cases the window shifts by 1 byte — the advance returned by the window delegate is ignored by the re-scan.

The re-scan covers retained data **from the start of the stream up to the boundary of the last direct emission**, advancing byte by byte at every position and ignoring the advance returned by the window delegate: after a success the direct pass jumps by the packet length and can skip the start of an overlapping valid packet (section 6) — only an exhaustive, gap-free pass closes such losses. Beyond the boundary the direct pass has not yet processed positions, so their repeated checking continues as ordinary scanning. This rules out double-confirmation of the same sectors beyond a single rebind. If the other-format scanner is idle (`Scan` calls are sequential), the accumulator is temporarily re-attached to it for a synchronous rebind and then returned to the active scanner. Intermediate rebinds of several headers do not conflict: each window is verified only against its own H5; confirmations of other slots are untouched.

9. File Assembly

Assembly (`ReceptionSlot.TryAssemble`) is the codec's most involved algorithm. The authenticity arbiter is the whole-file SHA-256 from H3. What follows is a working overview; the formal treatment (structure invariants, exact search-space definitions, the mathematics of the odometer and rotation,

worked examples, guarantee boundaries) is in `AssemblyGuide.Academic.en.md` (an engineering retelling — `AssemblyGuide.Engineer.en.md`).

9.1. Overall plan

```
TryAssemble
├─ selected combination = first (most confirmed) versions of every sector
├─ choice points = sectors with equally-probable versions (ChoicePoint)
│
├─ no choice points → a single attempt: direct assembly → RS
│
├─ choice points exist:
│   1. attempt #1 (the zero combination) with timing
│   2. combination count C = product of tie multiplicities
│   3. time estimate = t(attempt) · C
│   4. C ≤ 100,000 and estimate ≤ 30 s → exhaustive search (9.6)
│       otherwise → heuristic rotation (9.7)
│
└─ result = 1 and M > 0 → volume subset search (9.8)
```

9.2. A single attempt: direct assembly

`BuildPreferredSelection()` — an array of length N+M; for every received number `versions[0]` is taken (the most confirmed payload). Direct assembly succeeds when **all N data volumes** are present: the payloads are concatenated into an N·64 buffer, trimmed to `FileSize`, and verified by SHA-256 (9.9).

9.3. A single attempt: RS recovery

Direct assembly failed and `EccCount > 0`:

1. A validity map of length N+M (volume received/erased) and a slot array with the selected payloads are built.
2. If fewer than N volumes are received, there is nothing to recover from — fail.
3. `RsCodecAdapter.Decode(sectors, map, N)`: passthrough if data is intact; fail if erased data exceeds available ECC; otherwise the system is solved and erased data volumes are recovered symbol by symbol.
4. The recovered N volumes are concatenated, trimmed, and SHA-256-verified.

Note that RS recovery is already part of **every** attempt of the combination search — the search varies only the ambiguous sectors, while missing volumes are re-covered by ECC each time.

9.4. Choice points

A `ChoicePoint` is a sector where several versions share the **maximum** confirmation count (`TiedVariantCount` ≥ 2). Versions with lower counts do not take part in the branching: they are strictly worse confirmed and are selected by neither search strategy within the current call — if the truth ends up in the confirmation minority, assembly honestly refuses (the scenarios and the trade-off are covered in `AssemblyGuide.Academic.en.md`, sections 8 and 16).

9.5. Strategy selection

`SectorCombinationMath.CountCombinations(factors)` is the product of multiplicities, saturating at `long.MaxValue` (overflow is detected and reported as `> long.Max`). The first (zero) attempt is assembled with timing; the exhaustive-search estimate = attempt time $\times$ C. `ShouldUseExhaustiveSearch(C, estimate, options)` requires both: $C \leq \text{MaxExhaustiveCombinations}$ (100,000) and estimate $\leq \text{TimeBudget}$ (30 s). The limits are configurable via `SectorVersionSearchOptions`.

9.6. Exhaustive search

A Cartesian walk over all combinations of equally-probable versions. The selection indexes are held in `indexes[]` with moduli `moduli[]` (the choice-point multiplicities); `AdvanceIndexes` is a classic **odometer**: the lowest position on the right, increment with carry; on full exhaustion — false with a reset. The zero combination has already been checked, so the loop runs from 1 to C-1: apply the indexes to the selected set (`ApplyIndexes`), assemble (9.2 $\rightarrow$ 9.3), return on success. Each iteration checks cancellation and the 30-second time budget.

9.7. Heuristic rotation

When the exhaustive walk is unreasonable (C is large), the following observation is used: after each failed attempt, the first equally-probable variant of **every** ambiguous slot is moved to the end of the equally-probable part (`RotateTiedVariants`; versions with lower counts are untouched, so the sort stays valid).

Example: a slot with versions A/B/C (equal counts) and a slot X/Y yield the attempt sequence:

```
A+X → B+Y → C+X → A+Y → B+X → C+Y
```

The number of unique states of the synchronous rotation is the **LCM of the cycle lengths** (`CountRotationStates`), capped at `MaxHeuristicAttempts` (100,000). For the example: $\text{LCM}(3,2) = 6$ — indeed all 6 combinations are covered. For pairwise-coprime multiplicities the LCM equals C and the rotation is equivalent to exhaustive search; with common divisors some combinations are unreachable (the price of the heuristic), but the most likely "systematic" forgeries (the same shift in all slots) are covered. The time budget and cancellation are checked at every state.

9.8. Volume subset search

Stage 3 covers **silent corruption**: a sector is damaged, yet its number is intact and the truncated D3 hash matches; the version is the only one — no choice point, and RS with the full map takes the corrupted volume for the truth. `SubsetMaskPlanner` builds exclusion masks by levels: the base — all collision slots (excluded and recovered as a whole); level 1 — single exclusions of present volumes by suspicion (fewer confirmations first, ties pseudo-randomly with a seed from H5); levels 2–3 — pairs and triples from the shortlist. An attempt is RS recovery of the excluded volumes from the rest plus the final SHA-256; the limits are the cap on erased data volumes ($E \leq 32$), 100,000 attempts, 30 s. The formal treatment is in `AssemblyGuide.Academic.en.md` (section 12).

9.9. Result verification

`TrimAndVerify`: the N·64 buffer is trimmed to `FileSize` (the last volume was zero-padded), SHA-256 is computed and compared with H3. On mismatch the buffers are wiped and the attempt counts as failed. On match the file is returned: the assembly is cryptographically proven; there are no "partially correct" results.

10. The Randomized Test Bench

`DataShield.Demo` — a continuous stability and performance bench:

- input 1 byte ... 256 KB (log-uniform), ECC 1–200%;
- with 10% probability a multi-file stream, 3% an empty file;
- every run is damaged by a mask from `DamageBits` (sector losses beyond the ECC budget, truncations, junk, reordering, duplicates, winner forgeries, etc.);
- **PASS** — in-budget damage, restored bit-perfect; **WARN** — deliberate over-budget damage/forgery: the codec's refusal is correct; **FAIL** — a real codec defect;
- a ring table of recent iterations with accumulated statistics and encode/decode speeds.

The bench runs forever and is the primary regression tool: any FAIL behavior is a bug.

11. Numeric Summary

Parameter	Value
Packet / Base64 size	75 bytes / 100 chars, no padding
Volume payload	64 bytes = 32 GF(2 ¹⁶) symbols
Header fields	name 14 B + size 3 B + SHA-256 + M 2 B + H5 24 B
Maximum file (H2 field)	16,777,215 bytes
Maximum volumes (field)	$N + M \leq 65,535$ (~4.19 MB of data without ECC)
Default redundancy	ECC 10%, headers 3% (minimum 3 copies)
Checksums	truncated SHA-256 (24 B / 9 B); verification — SHA-256
Version-search limits	100,000 combinations/states, 30 s
Subset-search limits	100,000 masks, $E \leq 32$, 30 s